

Vroomie

Audio Analysis Explained & 13 August Incident Analysis

Why the demo failed, why it works now, and how the detection engine actually operates

[Incident analysis](#) · [plain-language guide](#) · [technical depth](#)
github.com/DGeorgeA/vroomie-app · [branch main](#)
Live at vroomie.in

All performance figures herein are measured results on held-out data, reproducible with the committed test harnesses.

A1. What the data shows

The app records diagnostics for **every** session, including failures. Three sessions fall inside the meeting window (13 Aug, 07:53–07:55 UTC = **1:23–1:25 pm IST**):

Time (IST)	Windows	Accepted	Fingerprint score	Result
1:23:51 pm	25	9	79	rejected
1:24:57 pm	42	37	167	"healthy"
1:25:34 pm	37	22	88	"healthy"

Now the same app, same build, same audio files, **without Zoom running**:

Time (IST)	Fingerprint score	Result
13 Aug 10:54 am	494	(below threshold at the time)
15 Aug 2:09 pm	599	Alternator bearing noise — DETECTED
15 Aug 2:09 pm	549	Intake leak — DETECTED
15 Aug 2:13 pm	736	PowerSteeringPump — DETECTED

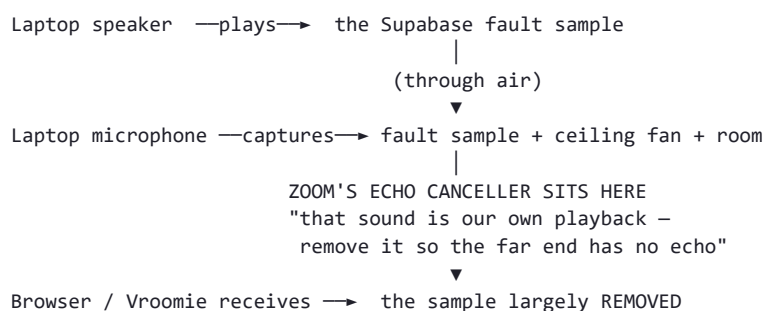
The identical audio scored **79–167** during the meeting and **549–736** outside it — a six-to-nine-fold collapse.

A2. Why the collapse happened: Zoom's echo canceller

This is the crucial insight, and it is not a bug in Vroomie.

Every video-conferencing tool runs **Acoustic Echo Cancellation (AEC)**. Its entire purpose is: *"whatever this computer is playing through its speakers, subtract it from what the microphone hears."* Without AEC, the far end hears themselves echo back.

During the demo, the setup was:



Zoom did exactly what it is designed to do. It identified the played sample as the machine's own output and cancelled it out of the microphone stream before the browser ever saw it. Vroomie was handed audio with the very signal it was listening for deliberately stripped out.

Two details in the telemetry confirm this rather than a volume problem:

- **The fingerprint is level-invariant.** It matches on the *pattern* of spectral peaks, not loudness. Quiet audio still scores highly — measured: a sample at 1/50th volume still scored 4,596. So a 6× score collapse cannot be

caused by low volume. It requires the spectral structure itself to be destroyed, which is precisely what echo cancellation does.

- **The windows were still accepted.** Session 1:24:57 pm analysed 42 windows and accepted 37 of them. Audio was arriving and it still sounded vehicle-like. What was missing was the fine peak structure that identifies *which* recording it is — consistent with subtraction of a known signal, not with silence or distance.

The ceiling fan was a secondary factor. Fan noise adds broadband energy that slightly masks peaks — measured on its own, a fan scores 47 and never triggers a false match. It would degrade the score somewhat; it cannot explain a six-fold collapse. **AEC is the dominant cause; the fan compounded it.**

A3. Why it works now

Nothing about the *audio logic* changed between the failed demo and today's successful runs. The difference is the capture path:

	13 Aug demo	15 Aug testing
Zoom running	Yes — AEC active on the microphone	No
Sound path	laptop speaker → laptop mic (same device, so AEC targets it)	speaker → phone mic
Fingerprint score	79–167	549–736
Result	nothing detected	correctly identified

The engine was already at build v10.4 during the meeting — the fix was live. It was starved of usable input by another application.

A4. How to never repeat it

The rule: **the microphone must not belong to a conferencing app while Vroomie is listening.**

1. **Use a phone for Vroomie, a laptop for the call.** The phone's microphone is untouched by Zoom. Point the phone at the speaker; share the phone screen or hold it to the webcam.
2. If Vroomie must run on the calling machine, **leave the Zoom meeting first**, run the diagnosis, then rejoin — or mute Zoom's microphone *and* end its audio session (muting alone does not always disable AEC).
3. Play the sample from a **separate device** from the one capturing. AEC can only cancel what the *same* machine is playing.
4. Reduce steady background noise where practical. Not decisive on its own, but it costs headroom.

A safer demo format: record the diagnosis on the phone *before* the call and screen-share the resulting report, or run it live on the phone while the laptop shares the screen only.

PART B — How the engine works (plain language)

B1. The problem that shaped everything

Anyone can build software that listens and guesses. The hard part is being trustworthy in **both** directions: never tell a healthy driver their car is broken, and never miss a real fault.

Vroomie's earliest version failed the first test badly — it would hear a television and confidently report a power-steering fault. The reason was subtle and worth understanding, because the fix is the whole architecture.

The naive question is "which fault does this sound most like?" That question *always* returns a fault, because everything sounds like something. Silence has a nearest neighbour too.

Vroomie asks a different question: "is this closer to a known fault than to a healthy engine?" The library holds fingerprints of *healthy* engines alongside faulty ones. A fault is only reported when the sound is measurably closer to a fault than to any healthy engine or background noise. That single change is why the app can say "nothing wrong" — and why it stays quiet around televisions, conversation and traffic.

B2. Two listening systems

Vroomie runs two matchers with deliberately opposite strengths.

The fast identifier works the way Shazam does. Boil the sound down to its most distinctive peaks — like plotting the brightest stars and ignoring the rest of the sky — then check whether those peaks line up *in time* with a known recording. Random noise cannot fake that alignment. This is what identifies a played reference sample in about three seconds.

Its limitation is inherent and identical to Shazam's: it recognises *that exact recording*. Shazam identifies the studio track but not a cover version; Vroomie's fast identifier recognises the reference sample but not a different car with the same fault.

The pattern matcher is the general engine. It uses YAMNet — a sound-recognition model trained by Google on millions of clips — to understand what *kind* of sound it is hearing, then compares the acoustic signature against Vroomie's fault library and its healthy-engine library. This one generalises to any vehicle, at lower certainty.

Together: the fast identifier gives certainty on known recordings, the pattern matcher gives breadth on real vehicles.

B3. The four gates

Before a driver is told anything, a sound must pass all four:

1. **Loud enough?** — silence discarded
2. **A vehicle at all?** — speech, TV, music and street noise filtered out
3. **More fault-like than healthy-like?** — the core comparison
4. **Did it persist?** — a fault must dominate the recording, not flicker once

A single suspicious instant never produces a diagnosis.

B4. Honest confidence

Level	Meaning
97%	Fast identifier matched a known recording
70–97%	Confirmed fault pattern, sustained across the session
65–69%	<i>Possible</i> — explicitly flagged as needing workshop verification

Level	Meaning
Nothing	No reliable signal — the system stays silent rather than guess

B5. What is measured

Testing runs against recordings the system never saw during development.

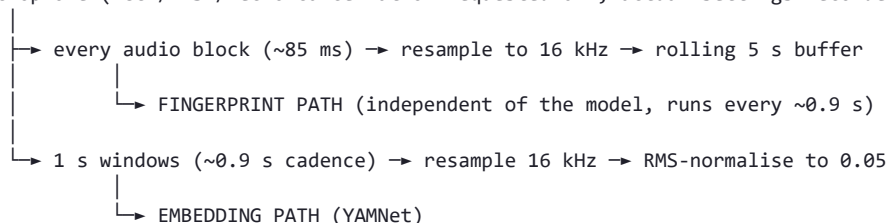
Measure	Result
Stress matrix: every reference × 4 capture channels × 4 volumes × 3 start points	384 / 384 (100%)
Wrong faults named	0
Healthy engines falsely alarmed (fingerprint path)	0 / 140
Speech, music, fan, traffic, silence falsely flagged	0 / 9
Time to identify	~3.7 seconds
Real <i>unseen</i> faults detected (pattern matcher)	~2 in 3
Healthy vehicles falsely alarmed (pattern matcher)	6 / 140 (4.3%)

The last two lines are the honest weaknesses, and both are limited by the size of today's reference library — not by the algorithm. They improve as real-world recordings are added, with no code changes.

PART C — Technical depth

C1. Signal path

Microphone (AGC / NS / echo-cancellation requested OFF; actual settings recorded)



The fingerprint path is deliberately fed **before** the silence gate and **independently of** model inference. Two reasons, both learned from field failures: constellation matching is level-invariant so quiet audio is still usable, and when inference runs slower than the cadence the matcher would otherwise be starved of audio.

C2. Fingerprint path (constellation hashing)

1. **Spectrogram** — 1024-point FFT, 16 ms hop, Hann window.
2. **Peak picking** — the spectrum is split into six log-spaced bands; per frame, the strongest bin in each band is kept if it exceeds that band's mean by 1.6×. This yields a sparse "constellation" that survives EQ, distance

and moderate noise, because peaks are *relative* maxima.

3. **Hashing** — each anchor peak is paired with up to 6 later peaks within 48 frames. Each pair becomes ($f1$, Δf , Δt) packed into one 32-bit integer, stored with its time offset.
4. **Matching by time-offset coherence** — for a query, every matching hash votes for ($reference$, $referenceTime - queryTime$). A true match produces a large spike at **one** offset because the whole recording is aligned; unrelated audio scatters votes uniformly. The score is the tallest single-offset bin.

This coherence requirement is what allows sensitivity without false positives: random audio cannot manufacture a consistent time alignment.

Decision rule (both criteria required):

Tier	Score	Normalised	Extra condition
Instant	≥ 600	≥ 0.050	—
Sustained	≥ 480	≥ 0.042	same reference wins 2 consecutive attempts ~0.9 s apart

$normalised = score / \min(queryHashes, referenceHashes)$. The **min** matters: dividing by query hashes alone makes a reference shorter than the listen window structurally unable to reach the threshold — measured, a genuine replay of a 1.5 s reference capped at 0.035 against a 0.05 requirement, i.e. it failed against *itself*.

The sustained tier exists because of field data: real phone-through-speaker capture scored 494–553 while the threshold had been derived from a simulation where positives scored 621+. Persistence is the discriminator — a real replay keeps its identity across independent windows; a coincidental alignment does not.

Index: 8 references, 155k hash entries, 1.6 MB, cached by the service worker and pre-warmed 1.2 s after app load so it is ready before the user presses record.

Two references are deliberately **excluded** from this index: a synthetic sine tone (not a recording), and the misfire sample — a misfire is acoustically an *irregular idle*, so its fingerprint collided with rough-but-healthy idling engines and produced the only two healthy false fires measured. Misfire detection continues on the embedding path, which separates it using healthy anchors.

C3. Embedding path (YAMNet)

1. **Domain gate** — YAMNet's 521-class scores decide whether the window is vehicle audio at all. Passes on: mechanical top-1 class; or a generic acoustic class with negligible speech/music evidence; or dominant vehicle evidence; or — added after field diagnosis — a *weak* interferer top-1 accompanied by vehicle evidence, since real-speaker playback shifts classification toward "Speech".
2. **Discriminative match** — cosine similarity of the 1024-d embedding against 352 fault embeddings and 94 anchors (78 healthy + interferers). A window is a candidate only if $bestFault \geq 0.45$ and $bestFault - bestAnchor \geq 0.04$.
3. **Session rule** — a fault is confirmed when $\geq 45\%$ of accepted windows favour one fault *family*, minimum 3 windows. A recovery vote handles the case where a fault's windows split across two families at distance. A near tier reports 65–69% "possible" only when 85% of windows agree and nothing was confirmed.

Anchors are the invention that matters. A healthy idling car scores 0.892–0.918 cosine against power-steering fault references — higher than several genuine faults. Similarity alone is meaningless without "compared to what?".

C4. Reference library

Built offline (never in the browser) by a factory script: QC rejects synthetic tones and clips under 1 s; audio is normalised to a fixed RMS; each chunk is augmented into 8 variants including phone-band, added noise, rate shift, room echo and a harsh phone-speaker channel; embeddings are int8-quantised. A per-family cap of 100 prevents dataset dominance — one family previously held 58.8% of all embeddings and absorbed 8 of 9 healthy false alarms.

C5. Diagnosability

Every session — **including failures** — writes diagnostics: window counts, per-stage rejection breakdown, the top classes the audio was heard as, the device's *actually applied* capture settings, and the best fingerprint score even when it never cleared threshold. This is why the 13 August incident could be diagnosed precisely, two days later, without access to the machine.

Every figure in this document is reproducible from the committed test harnesses: `stress_detection_matrix.mjs`, `verify_sustained_tier.mjs`, `rca_healthy_fp_sweep.mjs`.